

CASE STUDY #2

React Dashboard Performance Optimization

RevenuePulse – SaaS Revenue Analytics Dashboard

PROJECT OVERVIEW

RevenuePulse is a production-style React / Next.js SaaS analytics dashboard built for monitoring recurring revenue and subscription analytics.

The dashboard provides:

 Monthly Recurring Revenue (MRR)

 Revenue Trends

 Subscription Analytics

 Customer Metrics

 Retention Analytics

 Cohort Analysis

 Account & Team Management

As the dashboard grew in complexity and dataset size increased, users began noticing several frontend performance issues affecting responsiveness and overall usability.

THE PROBLEM

Users reported that the Revenue dashboard felt slow and heavy during normal usage.

Common feedback included:

 "Dashboard metrics take time to appear."

 "Revenue chart loads slowly."

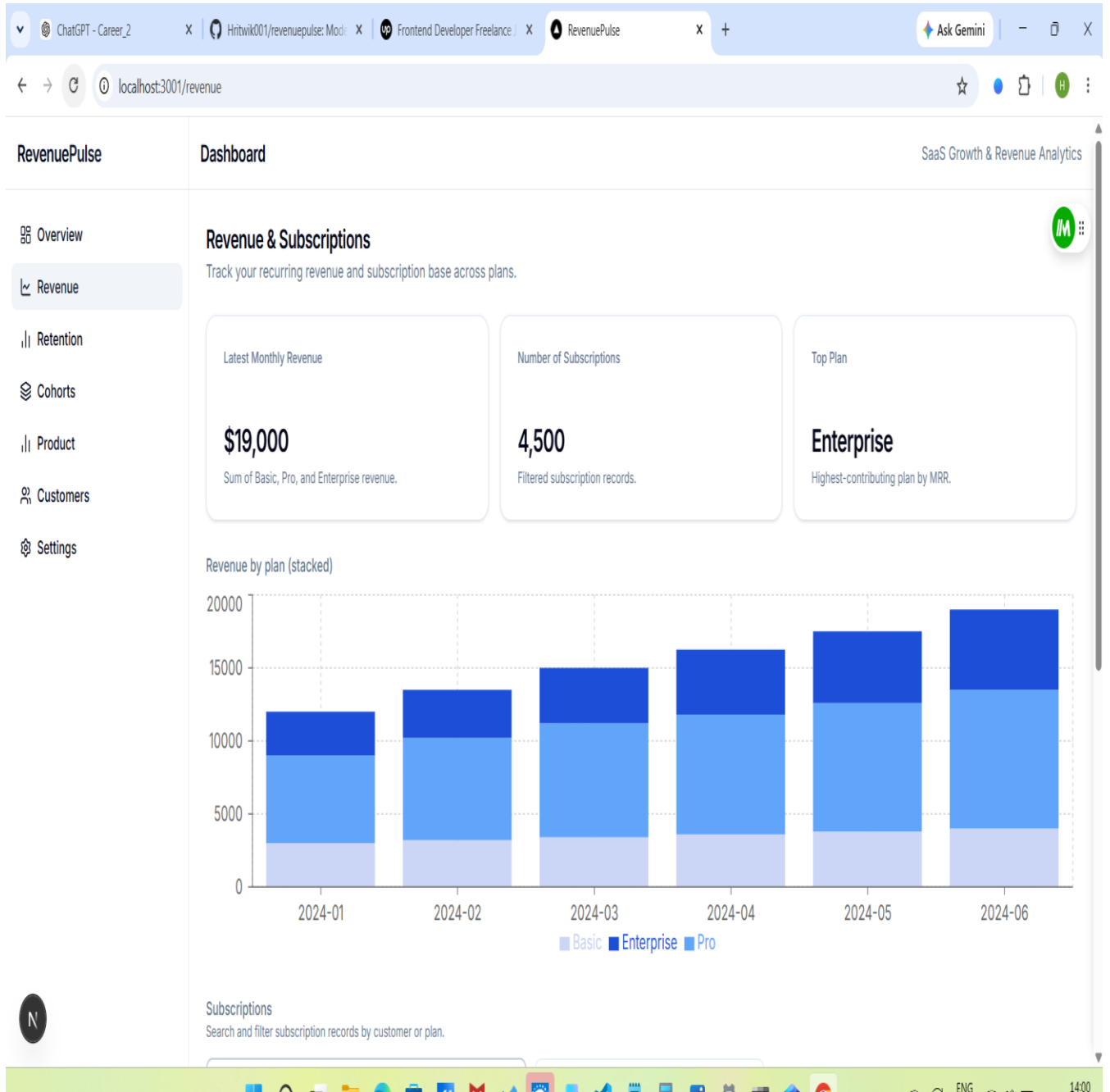
 "Searching subscriptions feels laggy."

 "Changing plan filters takes too long."

 "Dashboard refresh feels sluggish."

The objective of this investigation was to identify the root causes of these issues and systematically optimize the dashboard while preserving the existing functionality.

Revenue Dashboard



Dashboard Experience

Observed Issues

 KPI cards loaded slowly

 Revenue chart rendered with noticeable delay

 Search typing lag

 Slow subscription filtering

 Heavy dashboard refresh

Before

 [RevenuePulse - Before Optimization Demo](https://drive.google.com/file/d/11qBFfqrGdqf8lOB6x7Zv5_Jra0xtASo_r/view?usp=sharing)

https://drive.google.com/file/d/11qBFfqrGdqf8lOB6x7Zv5_Jra0xtASo_r/view?usp=sharing

After

 [RevenuePulse - After Optimization Demo](https://drive.google.com/file/d/1GBauEmln7Z4Mu3KuQSTaLvDPmEOv8Hu7/view?usp=sharing)

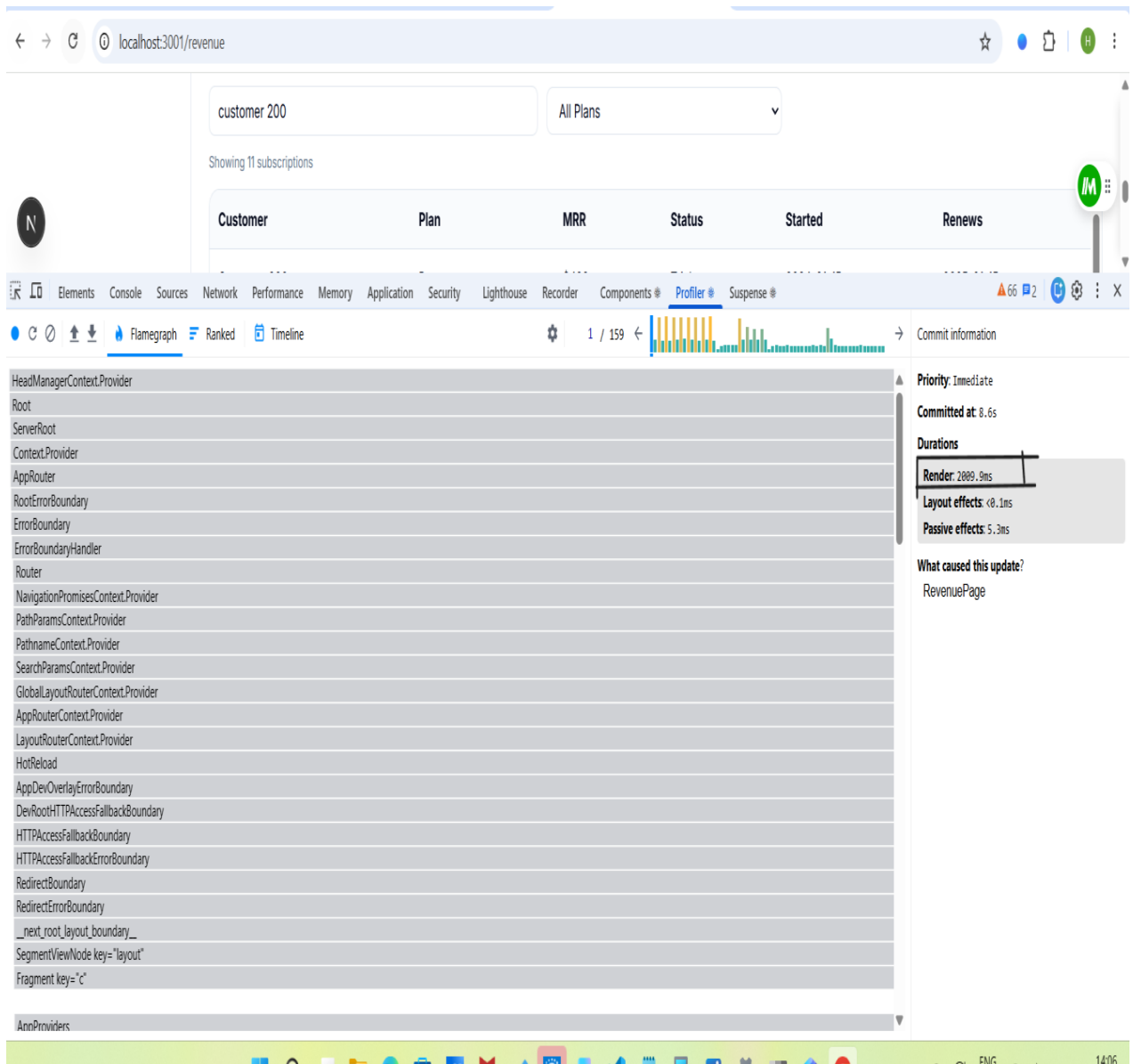
<https://drive.google.com/file/d/1GBauEmln7Z4Mu3KuQSTaLvDPmEOv8Hu7/view?usp=sharing>

React Profiler Analysis

Initial React Profiler recording immediately confirmed expensive rendering work.

Before

 Render Time: 2000ms

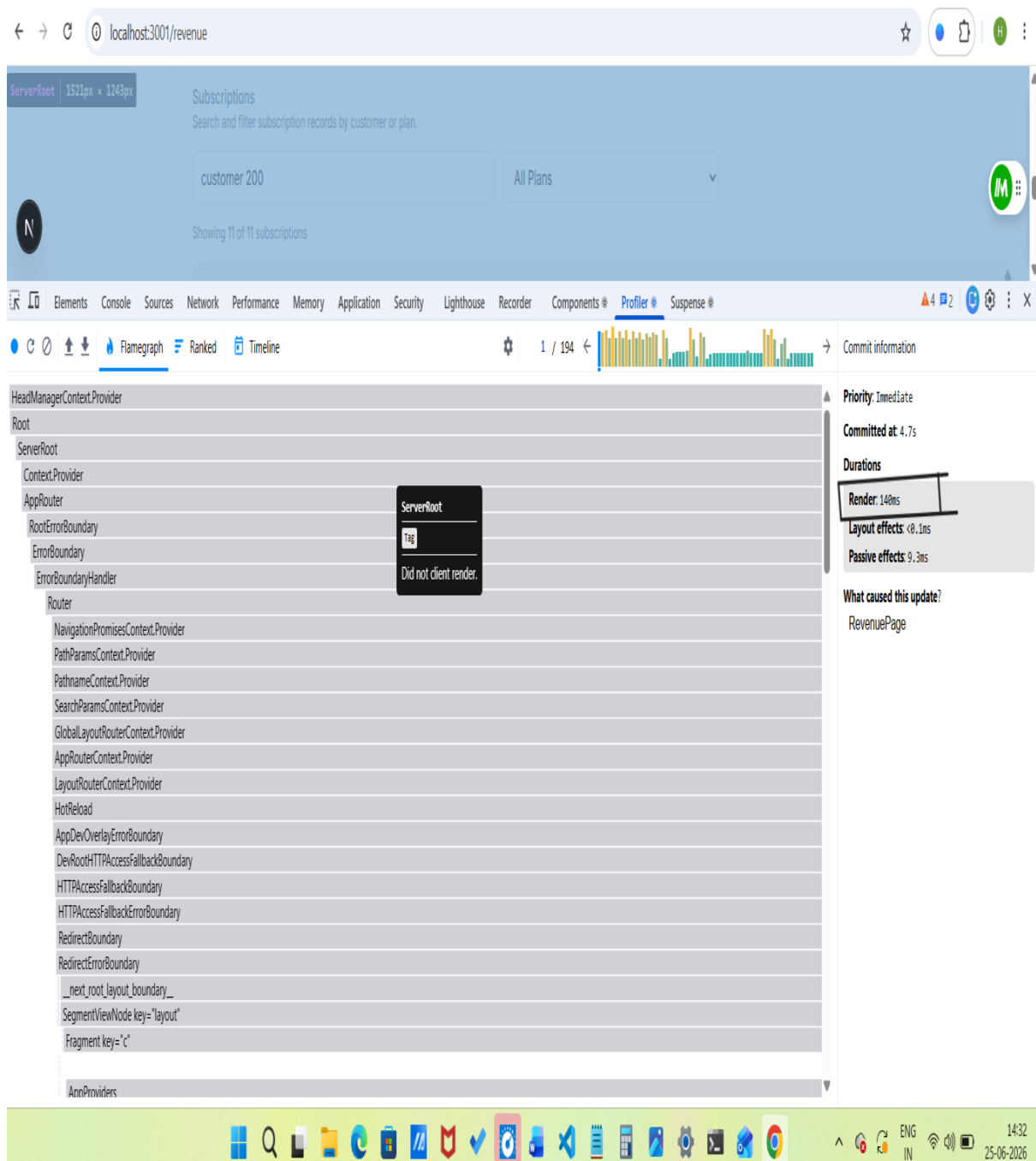


The dashboard required substantial rendering work before becoming responsive.



After

 Render Time: 140ms



Rendering became significantly faster after optimization.

INVESTIGATION APPROACH

Instead of immediately changing the code, a structured frontend performance engineering workflow was followed.

Problem



Evidence Collection



React Profiler Analysis



Root Cause Identification



Optimization



Validation

Every optimization was supported by measurable evidence.

ROOT CAUSE ANALYSIS

The investigation identified several independent bottlenecks.

PROBLEM #1

Slow KPI Card Rendering

Observation

Dashboard metrics appeared several seconds after the page loaded.

Root Cause

Dashboard metrics were intentionally delayed, creating unnecessary waiting before users could interact with important business information.

Impact

 Slower perceived dashboard loading

 Poor first impression

 Delayed visibility of key business metrics

PROBLEM #2

Slow Revenue Chart Rendering

Observation

Revenue analytics required several seconds before becoming visible.

Root Cause

The revenue visualization waited before rendering, forcing users to wait before viewing historical revenue data.

Impact



Delayed analytics visibility



Poor dashboard responsiveness



Slower analytical workflow

PROBLEM #3

Slow Filter Performance

Observation

Changing subscription plans produced a noticeable delay before the dashboard updated.

Example

All Plans



Enterprise



Dashboard Refresh



Chart Update



Table Update

Root Cause

Changing filters triggered unnecessary dashboard-wide refresh work and delayed updates.

Impact



Slow dashboard interaction



Delayed chart updates



Sluggish user experience

PROBLEM #4

Slow Search Experience

Observation

Typing into the subscription search field felt slow.
Each keystroke triggered expensive filtering work across
thousands of subscription records.

Example

c

cu

cus

cust

custom

customer

Each keystroke caused:

 Dataset filtering

 React rendering

 Dashboard updates

Impact

 Typing lag

 Slow search response

 Increased rendering work

OPTIMIZATION IMPLEMENTED

Several React performance optimizations were introduced.

OPTIMIZATION #1

Removed Artificial Dashboard Delays

Before

Delayed KPI rendering

Delayed dashboard initialization

After

Immediate KPI rendering

Immediate dashboard readiness

Benefit

 Faster first paint

 Better perceived performance

 Improved responsiveness

OPTIMIZATION #2

Optimized Revenue Chart Rendering

Before

Chart rendering delayed

After

Chart rendered immediately after data became available

Benefit

 Faster analytics visibility

 Improved dashboard responsiveness

OPTIMIZATION #3

Memoized Revenue Calculations

Implemented

 useMemo

Revenue calculations now execute only when underlying data changes.

Benefit

 Reduced repeated computation

 Lower rendering cost

 Better scalability

⚡ OPTIMIZATION #4

Memoized Subscription Filtering

Implemented

✓ useMemo

Filter operations execute only when search text or selected plan changes.

Benefit

⚡ Faster filtering

⚡ Reduced unnecessary computation

⚡ Lower CPU usage

⚡ OPTIMIZATION #5

Optimized Search

Implemented

✓ useDeferredValue

Search updates no longer compete directly with rendering.

Benefit

⚡ Smoother typing

⚡ Better interaction responsiveness

⚡ Improved user experience

OPTIMIZATION #6

Reduced Table Rendering

Before

Render entire subscription dataset

After

Render only visible subscription rows

Benefit

 Smaller DOM

 Lower rendering workload

 Better scalability

RESULTS

React Profiler Comparison

Before

 Render Time

2000ms

After

 Render Time

140ms

PERFORMANCE IMPROVEMENT

Before

 Slow dashboard

 Heavy rendering

 Delayed interactions

After

 Responsive dashboard

 Faster rendering

 Immediate interactions



BEFORE VS AFTER REACT PROFILER

Before



customer 200 All Plans

Showing 11 subscriptions

Customer	Plan	MRR	Status	Started	Renews
...	...	...	...	...	...

React DevTools Profiler

Commit information

Priority: Immediate

Committed at: 8.6s

Durations

Render: 2889.9ms

Layout effects: <0.1ms

Passive effects: 5.3ms

What caused this update?

RevenuePage



After



localhost:3001/revenue

Subscriptions

Search and filter subscription records by customer or plan.

customer 200 All Plans

Showing 11 of 11 subscriptions

Profiler

Commit information

Priority: Immediate

Committed at: 4.7s

Durations

Render: 140ms

Layout effects: <0.1ms

Passive effects: 9.3ms

What caused this update?

RevenuePage

ServerRoot

Tag

Did not client render.

HeadManagerContext.Provider

Root

ServerRoot

Context.Provider

AppRouter

RootErrorBoundary

ErrorBoundary

ErrorBoundaryHandler

Router

NavigationPromisesContext.Provider

PathParamsContext.Provider

PathnameContext.Provider

SearchParamsContext.Provider

GlobalLayoutRouterContext.Provider

AppRouterContext.Provider

LayoutRouterContext.Provider

HotReload

AppDevOverlayErrorBoundary

DevRootHTTPAccessFallbackBoundary

HTTPAccessFallbackBoundary

HTTPAccessFallbackErrorBoundary

RedirectBoundary

RedirectErrorBoundary

_next_root_layout_boundary_

SegmentViewNode key="layout"

Fragment key="c"

AppProviders

Windows taskbar



RESULTS SUMMARY

Metric	Before	After
Render Time	2000ms	140ms
KPI Loading	Delayed	Immediate
Chart Loading	Slow	Fast
Search Experience	Laggy	Smooth
Filter Response	Slow	Fast
Dashboard Responsiveness	Poor	Excellent

USER EXPERIENCE IMPACT

Before

 Slow dashboard startup

 Slow KPI rendering

 Chart loading delay

 Search lag

 Filter lag

After

 Instant dashboard metrics

 Immediate revenue analytics

 Smooth search

 Fast filtering

 Responsive dashboard

BUSINESS IMPACT

Optimizing frontend performance directly improves:

-  User productivity
-  Faster business insights
-  Better dashboard responsiveness
-  Improved scalability
-  Better overall user experience

By reducing unnecessary rendering work and optimizing dashboard computations, the RevenuePulse dashboard became significantly more responsive while maintaining the same functionality.

TECHNIQUES USED

-  React Profiler
-  useMemo
-  useDeferredValue
-  Memoized Computation
-  Rendering Optimization
-  Component Optimization
-  Large Dataset Optimization
-  Dashboard Performance Optimization

KEY LESSONS

Performance optimization is not achieved through one change.

The largest improvements came from:

- ✓ Measuring first
 - ✓ Identifying bottlenecks
 - ✓ Removing unnecessary work
 - ✓ Rendering less
 - ✓ Computing only when required
 - ✓ Validating improvements with React Profiler
-

FINAL OUTCOME

The RevenuePulse dashboard was transformed from a sluggish analytics experience into a significantly more responsive application through systematic React performance engineering.

By identifying bottlenecks, optimizing rendering, reducing unnecessary computations, and improving dashboard responsiveness, the application delivered a much smoother user experience while preserving all existing functionality.



Before

 [RevenuePulse - Before Optimization Demo](https://drive.google.com/file/d/11qBFfqrGdqf8lOB6x7Zv5_Jra0xtASor/view?usp=sharing)

https://drive.google.com/file/d/11qBFfqrGdqf8lOB6x7Zv5_Jra0xtASor/view?usp=sharing



After

 [RevenuePulse – After Optimization Demo](https://drive.google.com/file/d/1GBauEmln7Z4Mu3KuQSTaLvDPmEOv8Hu7/view?usp=sharing)

<https://drive.google.com/file/d/1GBauEmln7Z4Mu3KuQSTaLvDPmEOv8Hu7/view?usp=sharing>
